1. KAFKA & SPARK TUTORIAL

1.1 Overview

Apache Spark is a unified analytics engine for large-scale data processing that originated at UC Berkeley's AMPLab in 2009 and was first open-sourced in early 2010; it entered the Apache Incubator in 2013 and graduated as a Top-Level Project in February 2014.

Spark's evolution centers on an optimized DAG scheduler and resilient, in-memory primitives (RDDs) that power high-level APIs (Scala, Java, Python, R) and a rich library stack—Spark SQL, pandas API on Spark, MLlib, GraphX, and Structured Streaming—so teams can build ETL, interactive SQL, streaming, graph, and ML pipelines on one runtime.

Popularity has been sustained by a large contributor base and marquee deployments (e.g., Netflix, Yahoo, eBay), making Spark one of the most active open-source big-data projects with 1,000+ contributors from 250+ organizations.

In practice, Spark is used for batch and near-real-time data engineering, exploratory analytics, feature engineering and model training, and iterative graph workloads.

Compared with Hadoop MapReduce, Spark's DAG execution and caching avoid repeated disk I/O, yielding order-of-magnitude speedups on iterative and interactive workloads—early results reported up to 20–40× improvements—while unifying multiple processing modes under one API to reduce operational and cognitive overhead.

Against point solutions such as classic MapReduce/Hive for batch or Storm-style systems for streams, Spark's single engine and libraries minimize system sprawl while preserving scalability on clusters or cloud platforms.

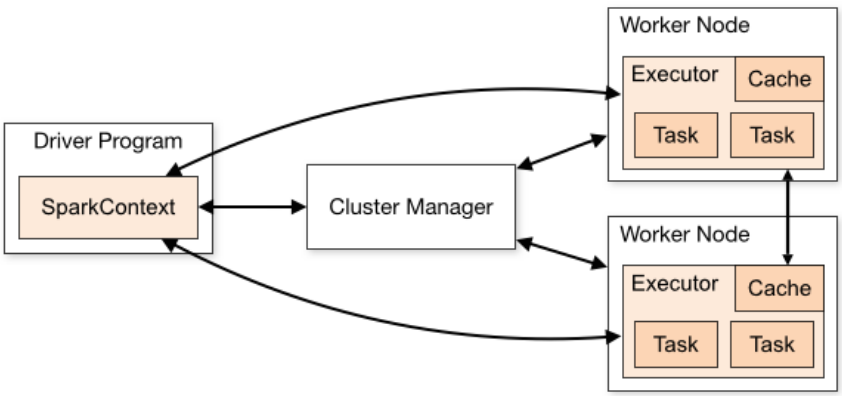

Figure 1: Apache Spark - Cluster Overview.

Apache Kafka is a distributed event-streaming platform that originated at LinkedIn around 2010 and was open-sourced in early 2011; it entered the Apache Incubator in 2011 and graduated a Top-Level Project on October 23, 2012. Recently, Kafka is widely

adopted—used by thousands of companies and over 80% of the Fortune 100—for high-throughput, low-latency pipelines and real-time analytics.

Kafka's design centers on an append-only, partitioned commit log and a pull-based consumption model with explicit offset control. This architecture enables horizontal scalability, durable retention, consumer replay, and strong ordering within partitions and properties relationship that power ETL, event sourcing, log aggregation, stream processing, and mission-critical data integration.

Compared with traditional message queues (e.g., RabbitMQ, ActiveMQ), Kafka excels at high-throughput event ingestion and multi-consumer replay: instead of deleting messages after delivery, Kafka retains them for a configurable period, allowing independent consumers to re-read streams at their own pace. In contrast, classic queues typically optimize for low-latency task dispatch and flexible routing but treat messages as transient, which complicates replay and large-scale fan-out.

Practically, that means: choose RabbitMQ/ActiveMQ for simple work queues and fine-grained routing; choose Kafka when you need a durable, scalable event backbone that feeds batch and streaming analytics, ML feature pipelines, and enterprise data meshes.

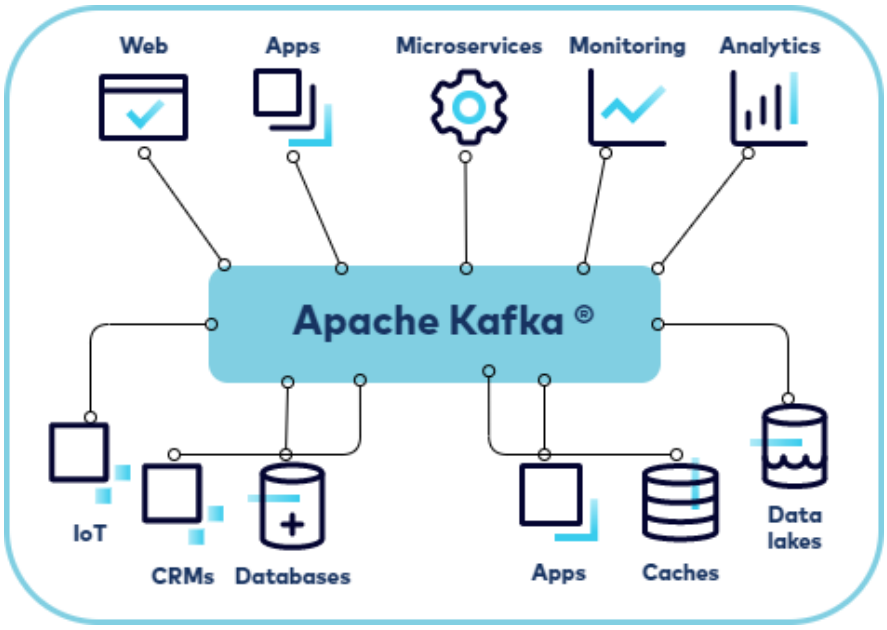

Figure 2: Apache Kafka - Pipeline Overview.

This first lab leverages Apache Spark and Apache Kafka to build practical fluency in three pillars: environment initialization, DataFrame-centric processing, and streaming ingestion from Kafka. You start a SparkSession and validate a working setup, then use DataFrames as the table-like abstraction for structured data while benefiting from the Catalyst optimizer's distributed execution planning. Finally, you connect to Kafka as a streaming source and reason about partitions, offsets, durability, replay, and checkpointing as you wire an end-to-end pipeline for low-latency analytics. By the end, you can move confidently from batch to continuous processing with consistent semantics across the stack.

2. TUTORIAL

2.1 Spark Initialization

To start a Spark job, you first prepare an application configuration and then create the runtime context that talks to your cluster. At a minimum, you choose where the job runs (local machine, Standalone master, YARN, or Kubernetes), give the job a name so you can find it in the Spark UI, size the driver and executors (memory and cores), and optionally cap the total cores your job can consume on a shared cluster.

Solution 1: In modern Spark, the recommended entry point is SparkSession, which owns an underlying SparkContext. Many teams set resource sizes at submit time (for example, with spark-submit flags) so the cluster manager enforces them consistently. In particular, the initialization steps are detailed as follows.

1. **Create the entry point (preferred)**
   - *Function*: `SparkSession.builder -> getOrCreate()`
   - *Purpose*: Start (or reuse) the application session; it owns the underlying `SparkContext`.
   - *Arguments*: none required here; add options via chained builder calls.
2. **Name the application**
   - *Function*: `appName(<app-name>)`
   - *Purpose*: Human-readable name shown in the Spark UI/logs.
   - *Arguments*: `<app-name>` (e.g., `"etl-users-daily"`).
3. **Choose where it runs (master / deploy target)**
   - *Function*: `master(<master-url-or-alias>)`
   - *Purpose*: Select local vs. cluster.
   - *Arguments*: `local[*]` (laptop), `spark://<host>:<port>` (Standalone), `yarn`, or a Kubernetes master alias.
4. **Set driver resources**
   - *Function*: `config("spark.driver.memory", <driver-memory>)`; optionally `config("spark.driver.cores", <driver-cores>)` (cluster mode).
   - *Purpose*: Give the driver enough RAM/CPU for planning and coordination.
   - *Arguments*: `<driver-memory>` (e.g., `"4g"`); `<driver-cores>` (e.g., `"2"`).
5. **Set executor resources**
   - *Function*: `config("spark.executor.memory", <executor-memory>)` and `config("spark.executor.cores", <executor-cores>)`
   - *Purpose*: Control per-executor capacity and task concurrency.

- *Arguments*: `<executor-memory>` (e.g., `"4g"`); `<executor-cores>` (e.g., `"4"`).

6. **Choose executor count (static)**
   - *Function*: `config("spark.executor.instances", <num-executors>)`
   - *Purpose*: Fixed pool size when not using dynamic allocation.
   - *Arguments*: `<num-executors>` (e.g., `"2"`).
7. **(Optional) Enable dynamic allocation**
   - *Function*: `config("spark.dynamicAllocation.enabled", <true-or-false>)`
   - *Purpose*: Let Spark grow/shrink executors based on backlog.
   - *Arguments*: `"true"` or `"false"`.
8. **(Standalone/Mesos) Cap total cores**
   - *Function*: `config("spark.cores.max", <max-total-cores>)`
   - *Purpose*: Prevent one job from consuming all CPU on shared clusters; coordinate with executor cores.
   - *Arguments*: `<max-total-cores>` (e.g., `"16"`).
9. **Add any other per-app properties**
   - *Function*: `config(<key>, <value>)`
   - *Purpose*: Pass additional Spark properties via the builder (propagated to `SparkConf`).
   - *Arguments*: key–value pairs (e.g., `"spark.sql.shuffle.partitions", "200"`).
10. **Finalize**
   - *Function*: `getOrCreate()`
   - *Purpose*: Materialize the configured `SparkSession` for SQL, DataFrame, and streaming APIs.

Code 1: Spark application initialization - Solution 1

```python
from pyspark import SparkContext, SparkConf
# Initialize SparkContext with a specific application name and master URL,
# and define driver's max core count, memory, and executor memory, cores, and instances.
conf = (SparkConf().setAppName("MyApp").setMaster("spark://10.1.11.5:7077")  # For local, replace with "local[x]" (x = # of cores, or * if use all)
    .set("spark.driver.memory", "4g")   # Max memory for driver, works for all CM
    .set("spark.driver.cores", "2") # Max # of cores for driver, works if CM = yarn/k8s. Not work if CM = standalone/mesos/local (use all cores by
      default)
    .set("spark.executor.memory", "4g")  # Max memory for each executor, works for all CM
    .set("spark.executor.cores", "4") # Max # of cores for each executor, works for all CM
    .set("spark.executor.instances", "2") # Max # of executors, works if CM = standalone/mesos. Not work if CM = yarn/k8s (use dynamic allocation by
      default)
    .set("spark.cores.max", "16")) # Max # of cores (global setting), must be divisible by spark.executor.cores, only works on standalone/mesos
sc = SparkContext(conf=conf).getOrCreate()
```

Common practice tip: For quick local checks, use `local[*]`. On clusters, many teams pass memory/cores at submit time (CLI or config files) so the scheduler enforces them consistently.

Solution 2: Using `SparkSession` is the modern, unified way to start a Spark application. You define the application name, choose where it runs (local or a cluster), set key resources for the driver and executors through builder configurations, and then finalize the session. `SparkSession.builder.getOrCreate()` will either return an existing session or create one from the options you set. It also owns the underlying `SparkContext`, so you can use SQL, DataFrame, and Streaming APIs from a single entry point.

1. **Create the entry point**
   - *Function*: `SparkSession.builder -> getOrCreate()`
   - *Purpose*: Start (or reuse) the application session; it embeds a `SparkContext`.
   - *Arguments*: none required at this step; options are added via chained builder calls.
2. **Name the application**
   - *Function*: `appName(<app-name>)`
   - *Purpose*: Human-readable name shown in the Spark UI and logs.
   - *Arguments*: `<app-name>` (e.g., `"etl-users-daily"`).
3. **Choose where it runs (master / deploy target)**
   - *Function*: `master(<master-url-or-alias>)`
   - *Purpose*: Select local vs. cluster; determines how resources are requested and scheduled.
   - *Arguments*: `local[*]` (laptop), `spark://<host>:<port>` (Standalone), `yarn`, or a Kubernetes alias.
4. **Set driver resources**
   - *Function*: `config("spark.driver.memory", <driver-memory>)` (and, in cluster mode, `config("spark.driver.cores", <driver-cores>)`)
   - *Purpose*: Give the driver enough RAM/CPU for planning and coordination.
   - *Arguments*: `<driver-memory>` such as `"4g"`; `<driver-cores>` such as `"2"`.
5. **Set executor resources**
   - *Function*: `config("spark.executor.memory", <executor-memory>)`, `config("spark.executor.cores", <executor-cores>)`
   - *Purpose*: Control per-executor capacity and task concurrency.
   - *Arguments*: `<executor-memory>` such as `"4g"`; `<executor-cores>` such as `"4"`.
6. **Choose executor count (static) or enable dynamic allocation**
   - *Function (static)*: `config("spark.executor.instances", <num-executors>)`
   - *Purpose*: Fixed pool size when not using dynamic allocation.
   - *Arguments*: `<num-executors>` such as `"2"`.

- *Function (dynamic)*: `config("spark.dynamicAllocation.enabled", <true-or-false>)`
- *Purpose*: Let Spark grow/shrink executors based on backlog.
- *Arguments*: `"true"` or `"false"`.

7. **(Standalone/Mesos) Cap total cores on shared clusters**

   - *Function*: `config("spark.cores.max", <max-total-cores>)`
   - *Purpose*: Prevent one job from consuming all CPU; coordinate with executor cores.
   - *Arguments*: `<max-total-cores>` such as `"16"`.

8. **Add any other per-app properties**

   - *Function*: `config(<key>, <value>)`
   - *Purpose*: Set additional Spark properties; these propagate to both `SparkConf` and the session.
   - *Arguments*: key–value pairs (e.g., `"spark.sql.shuffle.partitions", "200"`).

Code 2: Spark application initialization - Solution 2

```python
from pyspark.sql import SparkSession # = SparkContext + Spark's SQL + Spark Streaming +...
spark = (SparkSession.builder
    .appName("MyApp")  # Application name
    .master("spark://10.1.11.5:7077")  # Master URL
    .config("spark.driver.memory", "4g")   # same as above
    .getOrCreate()
)
```

2.2 DataFrame

A Spark DataFrame is a distributed table of data organized into named columns—conceptually like a SQL table or pandas DataFrame—with richer, built-in optimizations, and it's the primary structured API in Spark (available in Python, Scala, Java, and R). DataFrame queries are optimized by Spark SQL's Catalyst (logical/physical plan optimization) and executed efficiently by Tungsten, which together deliver substantial performance improvements over lower-level RDD code for many workloads. DataFrames power both batch and streaming analytics through the same APIs (e.g., via Structured Streaming), letting you load from many sources, transform with relational operations (select, filter, join, aggregate), and write to a wide range of sinks.

1. Generation

Code 3: DataFrame Generation Example

```
data = [("Abraham", "Apples", 5),
        ("Jacob", "Bananas", 3),
        ("Jacob", "Oranges", 2),
        ("Benjamin", "Apples", 2),
        ("David", "Bananas", 2),
        ("David", "Oranges", 5),
        ("Esau", "Apples", 3),
        ("Esau", "Bananas", 5),
        ("Ishmael", "Oranges", 4),
        ("Samael", "Apples", 4),
        ("Samael", "Bananas", 1),
        ("Samael", "Oranges", 1)]
columns = ["Customer", "Item", "Quantity"]
df = spark.createDataFrame(data, columns)
```

2. Visualization

Code 4: DataFrame Generation Example

```
df.show()  # Show the dataframe, similar to pandas df.head()
print(df.count())
df.printSchema()  # Print the schema of the dataframe, similar to pandas df.info()
```

```
+--------+-------+--------+
|Customer|   Item|Quantity|
+--------+-------+--------+
| Abraham| Apples|       5|
|   Jacob|Bananas|       3|
|   Jacob|Oranges|       2|
|Benjamin| Apples|       2|
|   David|Bananas|       2|
|   David|Oranges|       5|
|    Esau| Apples|       3|
|    Esau|Bananas|       5|
| Ishmael|Oranges|       4|
|  Samael| Apples|       4|
|  Samael|Bananas|       1|
|  Samael|Oranges|       1|
+--------+-------+--------+

12
root
 |-- Customer: string (nullable = true)
 |-- Item: string (nullable = true)
 |-- Quantity: long (nullable = true)
```

Figure 3: Visualization Outputs.

3. Aggregation operations

- Totally, how many apples are sold? (3 approaches)

Code 5: Sum Aggregation Example

```
from pyspark.sql.functions import col, sum

# APPROACH 1: Filter apple purchases -> aggregate by quantity, collect (the distributed result) to the driver node
total_apples = df.filter(col("Item") == "Apples").agg(sum("Quantity").alias("Total_Apples")).collect()[0]["Total_Apples"
]
print(f"Total Apples sold: {total_apples}")

# APPROACH 2: Group by item -> aggregate by quantity -> filter apple purchases, collect (the distributed result) to the
driver node
total_apples_2 = df.groupBy("Item").agg(sum("Quantity").alias("Total_Quantity")).filter(col("Item") == "Apples").collect
()[0]["Total_Quantity"]
print(f"Total Apples sold (approach 2): {total_apples_2}")

# APPROACH 3 & 4: Same as the first approach, but use SQL syntax
df.createOrReplaceTempView("purchases")  # Create a temporary view for SQL queries
total_apples_3 = spark.sql("SELECT SUM(Quantity) AS Total_Apples FROM purchases WHERE Item = 'Apples'").collect()[0]["
Total_Apples"]
print(f"Total Apples sold (approach 3): {total_apples_3}")

# APPROACH 4: Same as the second approach, but use SQL syntax
total_apples_4 = spark.sql("SELECT Item, SUM(Quantity) AS Total_Quantity FROM purchases GROUP BY Item HAVING Item = '
Apples'").collect()[0]["Total_Quantity"]
print(f"Total Apples sold (approach 4): {total_apples_4}")
```

- Generate and join a price table with the purchase one.

Code 6: Generation of price table.

```
price_data = [("Apples", 1.0),
              ("Bananas", 0.5),
              ("Oranges", 0.8)]
price_columns = ["Item", "Price"]
price_df = spark.createDataFrame(price_data, price_columns)
price_df.show() # Show table to check content
```

Let's answer the question *How much did Jacob spend on groceries?*

Code 7: Joining price table with the purchase one.

```
from pyspark.sql.functions import expr
# Join purchases with prices on Item column
joined_df = df.join(price_df, on="Item", how="inner")
# Calculate total cost for each purchase
joined_df = joined_df.withColumn("Total_Cost", expr("Quantity * Price"))
# Filter for Jacob's purchases and sum the total cost
jacob_total = joined_df.filter(col("Customer") == "Jacob").agg(sum("Total_Cost").alias("Jacob_Total")).collect()[0]["
Jacob_Total"]
print(f"Jacob's total grocery spending: ${jacob_total:.2f}")
```

The results will be returned to $3.10

2.3 MinIO

MinIO is an open-source, high-performance, S3-compatible object store that runs anywhere (on-prem, cloud, Kubernetes), designed for scalable analytics/AI and large unstructured datasets. It provides durability and performance features typical of modern object storage—S3 API compatibility, distributed deployments, and erasure coding for data protection—making it a lightweight alternative to HDFS or proprietary appliances.

With Spark, MinIO is often the practical storage layer because Spark can read/write to any S3-compatible store via the S3A connector: you set the endpoint/credentials and use s3a://... paths, enabling a decoupled "compute on Spark, store on MinIO" architecture across local, hybrid, or Kubernetes clusters.

Let's practice saving data on MinIO. Firstly, we configure to make Spark's executors connect with MinIO as the followings.

Code 8: Configuration for the connection of Spark & MinIO.

```
spark.sparkContext._jsc.hadoopConfiguration().set("spark.jars.packages","org.apache.hadoop:hadoop-aws:3.4.1") # Library for connecting to AWS S3
  /MinIO
spark.sparkContext._jsc.hadoopConfiguration().set("fs.s3a.endpoint", "http://10.1.11.5:9000") # MinIO endpoint (location)
spark.sparkContext._jsc.hadoopConfiguration().set("fs.s3a.access.key", "test") # username
spark.sparkContext._jsc.hadoopConfiguration().set("fs.s3a.secret.key", "hcmuthpcc") # password
spark.sparkContext._jsc.hadoopConfiguration().set("fs.s3a.path.style.access", "true") # Enable path style access (http://10.1.11.5:9000/big-data
  /test/... instead of http://big-data.endpoint/test/...)
spark.sparkContext._jsc.hadoopConfiguration().set("fs.s3a.aws.credentials.provider", "org.apache.hadoop.fs.s3a.SimpleAWSCredentialsProvider") #
  Use basic credentials that we set above
spark.sparkContext._jsc.hadoopConfiguration().set("fs.s3a.impl", "org.apache.hadoop.fs.s3a.S3AFileSystem")  # Use S3A file system
spark.sparkContext._jsc.hadoopConfiguration().set("fs.s3a.committer.name", "magic")                        # Use magic committer, better than
  default FileOutputCommitter
spark.sparkContext._jsc.hadoopConfiguration().set("fs.s3a.committer.magic.partitioned.enabled", "true")     # Enable partitioned writing
spark.sparkContext._jsc.hadoopConfiguration().set("fs.s3a.fast.upload", "true")                             # Enable fast upload
spark.sparkContext._jsc.hadoopConfiguration().set("fs.s3a.fast.upload.buffer", "disk")                      # Use disk buffer for fast upload
  # OPTIONAL
spark.sparkContext._jsc.hadoopConfiguration().set("fs.s3a.committer.staging.conflict-mode", "replace")      # Overwrite existing files
  # OPTIONAL
spark.sparkContext._jsc.hadoopConfiguration().set("fs.s3a.committer.staging.abort.pending.uploads", "true") # Abort pending uploads
```

Secondly, data is written to MinIO storage by the following command.

Code 9: Write data to MinIO's storage.

```
df.write.mode("overwrite").csv("s3a://big-data/test/jacob.csv", header=True)
```

Notice: Add Spark Context on the go, add MinIO, and change committer to "magic" for better performance.

Warning: If config is set directly in hadoop, do not add spark.hadoop prefix, only add it during SparkConf/SparkSession initialization.

3. EXERCISES

This exercises is leveraged to help students practice Spark and Kafka operations professionally. The demands of exercise are shown as the followings.

- Get data from Kafka from 3 brokers according to 3 IP addresses: 10.1.1.10:30090, 10.1.1.27:30091, and 10.1.1.203:30092. The error may appear, which is caused by data format in Kafka. Resolve it!
- Get top 5 movies (by Id) by average rating, whose rating count is higher than 30.
- Get 5 worst tags (associated with lowest average rating).
- Does the answer for the above question mean movies with certain tags tend to receive lower ratings? For each movie associated with these tags, check their average ratings and how many movies have these tags.

Submission: The works must be done in a group of at most 3.

- Compress all necessary files (sources and a report) as **CO3137_BigDataLab_Lab01_<Student ID-1>_<Student ID-2>_<Student ID-3>.zip**.
- Students must submit and demonstrate their results before the end of the class.
- Early submitters (within the lab class) get bonus points. The rest can still get a 10 if they deserve it comprehensively.

4. Appendix: Local Kafka deployment

Note: In the event of Kafka broker's failure or if you want to run everything locally for debugging, you will need to have the following components installed in your machine.

- Python with pyspark version >= 3.5.0, latest version preferred, and kagglehub installed.
- Docker Desktop installed.

4.1 Deploy a Kafka cluster

Define the docker-compose.yaml file at your_directory_name directory:

```
services:
  broker-1:
    image: bitnami/kafka:3.6.0
    ports:
      - 9092:9092
    volumes:
      - broker-1-data:/bitnami/kafka
    environment:
      # The shared cluster identity that all brokers must agree on
      - KAFKA_KRAFT_CLUSTER_ID=ahihidongoc

      # This broker's unique node number in the cluster
      - KAFKA_CFG_NODE_ID=1

      # This node participates as both controller and broker
      - KAFKA_CFG_PROCESS_ROLES=controller,broker

      # The complete roster of all voting members
      - KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=1@broker-1:9093,2@broker-2:9093,3@broker-3:9093

      # Define the network interfaces this broker listens on
      - KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093,INTERNAL://:9094

      # Tell clients and other brokers how to reach this specific broker
      - KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9092,INTERNAL://broker-1:9094

      # Map listener names to their security protocols
      - KAFKA_CFG_LISTENER_SECURITY_PROTOCOL_MAP=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,INTERNAL:PLAINTEXT

      # The controller listener is used only for consensus operations
      - KAFKA_CFG_CONTROLLER_LISTENER_NAMES=CONTROLLER

      # Brokers communicate with each other using the internal listener
      - KAFKA_CFG_INTER_BROKER_LISTENER_NAME=INTERNAL
    profiles:
      - kafka

  broker-2:
    image: bitnami/kafka:3.6.0
    ports:
      - 9192:9092
    volumes:
      - broker-2-data:/bitnami/kafka
    environment:
      # Critical: Same cluster ID as broker-1
      - KAFKA_KRAFT_CLUSTER_ID=ahihidongoc
      - KAFKA_CFG_NODE_ID=2
      - KAFKA_CFG_PROCESS_ROLES=controller,broker
      - KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=1@broker-1:9093,2@broker-2:9093,3@broker-3:9093
      - KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093,INTERNAL://:9094
      - KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9192,INTERNAL://broker-2:9094
      - KAFKA_CFG_LISTENER_SECURITY_PROTOCOL_MAP=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,INTERNAL:PLAINTEXT
      - KAFKA_CFG_CONTROLLER_LISTENER_NAMES=CONTROLLER
      - KAFKA_CFG_INTER_BROKER_LISTENER_NAME=INTERNAL
    profiles:
      - kafka

  broker-3:
    image: bitnami/kafka:3.6.0
```

```
    ports:
      - 9292:9092
    volumes:
      - broker-3-data:/bitnami/kafka
    environment:
      # Critical: Same cluster ID as brokers 1 and 2
      - KAFKA_KRAFT_CLUSTER_ID=ahihidongoc
      - KAFKA_CFG_NODE_ID=3
      - KAFKA_CFG_PROCESS_ROLES=controller,broker
      - KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=1@broker-1:9093,2@broker-2:9093,3@broker-3:9093
      - KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093,INTERNAL://:9094
      - KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9292,INTERNAL://broker-3:9094
      - KAFKA_CFG_LISTENER_SECURITY_PROTOCOL_MAP=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,INTERNAL:PLAINTEXT
      - KAFKA_CFG_CONTROLLER_LISTENER_NAMES=CONTROLLER
      - KAFKA_CFG_INTER_BROKER_LISTENER_NAME=INTERNAL
    profiles:
      - kafka

volumes:
  broker-1-data:
  broker-2-data:
  broker-3-data:
```

Deploy the cluster:

```
$ docker compose --profile kafka up -d
```

4.2 Install Python-related packages

Create the Python virtual environment.

```
$ python -m venv your_directory_name
$ source your_directory_name/bin/activate
```

To download the forementioned packages, run the command:

```
$ pip install pyspark==4.1.0 kagglehub confluent-kafka jupyter-lab
```

Run the Jupyter notebook, or ideally, use VS Code's Jupyter extension:

```
$ jupyter lab
```

Run some setup code:

Code 10: Initialization for Local Development.

```
import kagglehub
from confluent_kafka.admin import AdminClient, NewTopic
from pyspark.sql import SparkSession

spark = (SparkSession.builder.appName("MyApp")
.config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-10_2.13:4.0.1,org.apache.kafka:kafka-clients:3.6.0,org.apache.spark:spark-
      streaming-kafka-0-10_2.13:4.0.1")
.master("local[*]")
.getOrCreate())
```

Download the dataset from Kaggle:

Code 11: Download Kaggle dataset.

```
path = kagglehub.dataset_download("grouplens/movielens-latest-small")
df_ratings = spark.read.csv(path + "/ratings.csv", header=True, inferSchema=True)
df_movies = spark.read.csv(path + "/movies.csv", header=True, inferSchema=True)
df_tags = spark.read.csv(path + "/tags.csv", header=True, inferSchema=True)
df_ratings.show(1)
df_movies.show(1)
df_tags.show(1)
```

Create Kafka topics:

Code 12: Create topics.

```
# First, delete topics if they exist
admin_client = AdminClient({'bootstrap.servers': 'localhost:9092'})
admin_client.delete_topics(['ratings', 'movies', 'tags'], operation_timeout=10)
topic_metadata = admin_client.list_topics(timeout=10)
print("Deleted if exists")

# Create new topics
new_topics = [NewTopic(topic="ratings", num_partitions=1, replication_factor=3),
              NewTopic(topic="movies", num_partitions=1, replication_factor=3),
              NewTopic(topic="tags", num_partitions=1, replication_factor=3)]
fs = admin_client.create_topics(new_topics)
for topic, f in fs.items():
    try:
        f.result()  # The result itself is None
        print(f"Topic {topic} created")
    except Exception as e:
        print(f"Failed to create topic {topic}: {e}")
```

Load the dataset to Kafka:

Code 13: Push data to Kafka using Spark.

```
# Push the dataframes to kafka broker
df_ratings.selectExpr("to_json(struct(*)) AS value") \
    .write \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("topic", "ratings") \
    .save()
df_movies.selectExpr("to_json(struct(*)) AS value") \
    .write \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("topic", "movies") \
    .save()
df_tags.selectExpr("to_json(struct(*)) AS value") \
    .write \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("topic", "tags") \
    .save()
# Check if data is in Kafka topics, however it's stored in binary format, and in "value" column
df_ratings_kafka = spark.read \
  .format("kafka") \
  .option("kafka.bootstrap.servers", "localhost:9092") \
  .option("subscribe", "ratings") \
  .load()

df_ratings_kafka = df_ratings_kafka.selectExpr("CAST(value AS STRING)")
df_ratings_kafka.show(1)
```